Задачи на отрезках

Закарлюка Иван Владимирович

Санкт-Петербургский государственный университет

Оглавление

1. Дерево отрезков 3

2. Дерево Фенвика 15

3. Массовые операции 21

Дерево отрезков

Мотивация

Задача: есть массив a длины n. Нужно:

- менять значение $a[i]$
- быстро вычислять функцию F на отрезке $[l, r]$ (например, сумму, минимум, максимум)

Наивное решение: обновление $O(1)$, запрос $O(n)$

Префиксные суммы: обновление $O(n)$, запрос $O(1)$ на префиксах

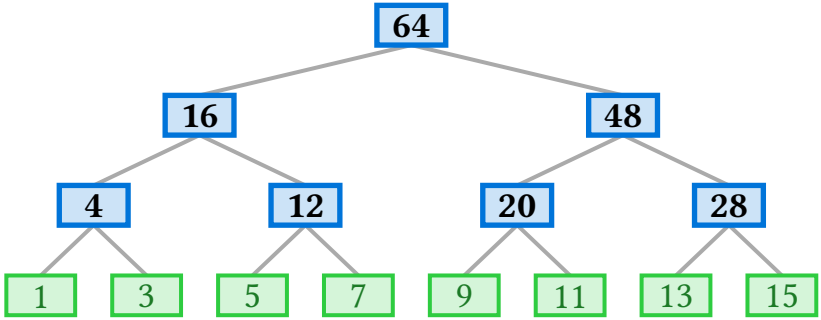
Разреженные таблицы: обновление $O(n)$, запрос $O(\log n)$

Нужна структура данных с $O(\log n)$ на обе операции.

Структура дерева отрезков

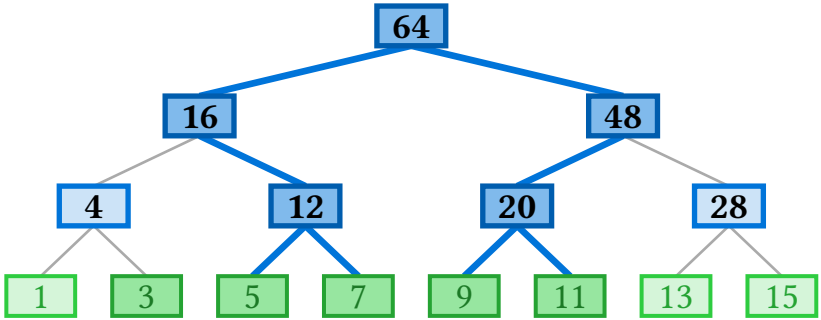
Дерево отрезков — двоичное дерево, узлы которого хранят результаты применения операции к своим поддеревьям.

Пример для массива $a = [1, 3, 5, 7, 9, 11, 13, 15]$:



Пример: запрос sum(2, 6)

Запросим сумму элементов $a_2 + a_3 + a_4 + a_5 + a_6 = 3 + 5 + 7 + 9 + 11 = 35$



Реализация

Хранить дерево отрезков проще всего в массиве t размера $4 * n$. Корень дерева = $t[1]$. Левый ребенок вершины $v - t[2 * v]$, правый $-t[2 * v + 1]$.

Построение

```
void build(int v, int l, int r) {
    if (l == r) {
        tree[v] = a[l];
        return;
    }
    int mid = (l + r) / 2;
    build(v*2, l, mid);
    build(v*2+1, mid+1, r);
    tree[v] = merge(tree[v*2], tree[v*2+1]);
}
```

Запрос на отрезке

```
int query(int v, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return neutral;  // не пересекаются
    if (ql <= l && r <= qr) return tree[v]; // полностью внутри
    int mid = (l + r) / 2;
    return merge(query(v*2, l, mid, ql, qr),
                 query(v*2+1, mid+1, r, ql, qr));
}
```

Рекурсия спускается только по нужным путям — $O(\log n)$

Обновление элемента

```
void update(int v, int l, int r, int pos, int val) {
    if (l == r) {
        tree[v] = val;
        return;
    }
    int mid = (l + r) / 2;
    if (pos <= mid)
        update(v*2, l, mid, pos, val);
    else
        update(v*2+1, mid+1, r, pos, val);
    tree[v] = merge(tree[v*2], tree[v*2+1]);
}
```

Время: $O(\log n)$

Итеративная реализация

Дерево отрезков можно реализовать и без рекурсии, что обычно будет немного быстрее

Итеративная реализация: построение

```
void build(const vector<int>& a, int n) {
    for (int i = 0; i < n; i++)
        tree[n + i] = leafs[i];
    for (int i = n - 1; i > 0; i--)
        tree[i] = tree[i * 2 + 1] + tree[i * 2];
}
```

Время: $O(n)$

Итеративная реализация: запрос

```
int query(int l, int r) {
    l += n;
    r += n;
    long long ans = 0;
    while (l < r) {
        if (l & 1 == 1)
            ans += tree[l++];
        if (r & 1 == 1)
            ans += tree[--r];
        l >>= 1; r >>= 1;
    }
    return ans;
}
```

Идея: два указателя l и r идут навстречу друг другу

Итеративная реализация: обновление

```
// присвоить a[pos] = val
void update(int pos, int val) {
    pos += n;
    tree[pos] = val;
    for (v >>= 1; v > 0; v >>= 1)
        tree[v] = tree[v * 2] + tree[v * 2 + 1];
}
```

Время: $O(\log n)$

Дерево Фенвика

Идея

Дерево Фенвика — альтернатива дереву отрезков

Идея: заведем некоторую функцию $F(i) \leq i$, тогда дерево Фенвика - массив t, $t_i = \Sigma_{k=F(i)}^{i} a_k$

То есть t_i отвечает за сумму на $[F(i), i]$

Тогда сумма на префиксе $\text{sum}(r) = t_r + \text{sum}(F(r) - 1)$

Будем рассматривать вариант `F(i) = x - (x & -x) + 1`. `x & -x` это младший бит x. Значит мы просто зануляем младший бит x применением фукнции.

Структура

тут должна была быть картинка, но я её не нарисовал.

Реализация

```
// добавить val к a[pos]
void add(int pos, int val) {
    for (; pos <= n; pos += pos & -pos)
        t[pos] += val;
}
```

Реализация суммы

```
// сумма на [1, pos]
int sum(int pos) {
    int res = 0;
    for (; pos > 0; pos -= pos & -pos)
        res += t[pos];
    return res;
}

// запрос на [l, r]
int range_sum(int l, int r) {
    return sum(r) - sum(l - 1);
}
```

Время: $O(\log n)$ на обе операции

Построение

```
void build() {
    for (int i = 1; i <= n; i++)
        t[i] = a[i];
    for (int i = 1; i <= n; i++) {
        int j = i + (i & -i);
        if (j <= n)
            t[j] += t[i];
    }
}
```

Время: $O(n)$

Альтернатива: добавлять элементы по одному через `add()` — тоже $O(n)$

Массовые операции

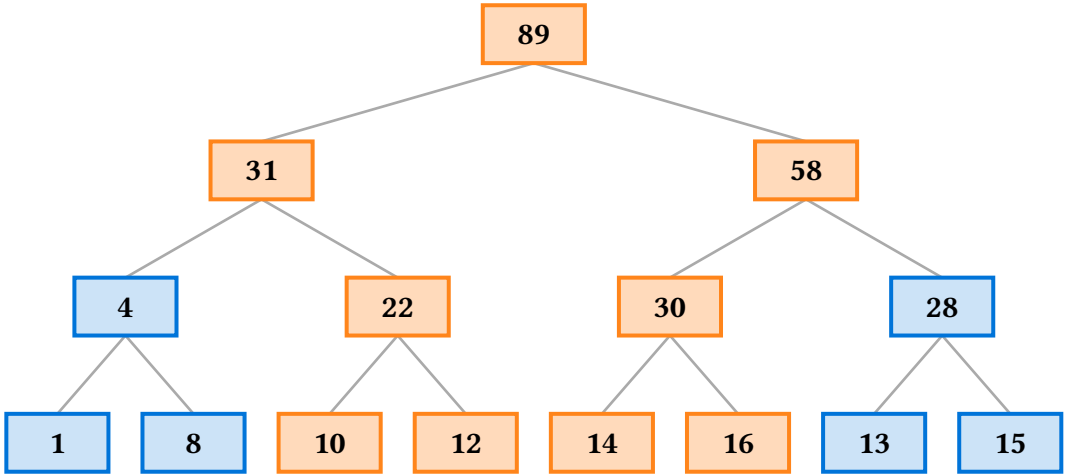
Проблема

Базовая версия: обновление одного элемента за $O(\log n)$

Новая задача: обновление целого отрезка за $O(\log n)$

```
// прибавить val ко всем элементам на [l, r]
void range_add(int l, int r, int val);
```

Пример



Отложенные операции

Идея: не применяем обновление сразу ко всем детям, а запоминаем его.

Каждый узел хранит:

- `tree[v]` — значение на отрезке $[l, r]$
- `lazy[v]` — отложенная операция, которую нужно применить к детям

Когда спускаемся в ребёнка — применяем накопленную операцию

Реализация: push

```
void push(int v) {
    if (lazy[v] != 0) {
        tree[v*2] += lazy[v];
        lazy[v*2] += lazy[v];

        tree[v*2+1] += lazy[v];
        lazy[v*2+1] += lazy[v];

        lazy[v] = 0;
    }
}
```

Реализация: range update

```
void update(int v, int l, int r, int ql, int qr, int val) {
    if (ql > r || qr < l) return;
    if (ql <= l && r <= qr) {
        tree[v] += val * (r - l + 1);
        lazy[v] += val;
        return;
    }
    push(v);
    int mid = (l + r) / 2;
    update(v*2, l, mid, ql, qr, val);
    update(v*2+1, mid+1, r, ql, qr, val);
    tree[v] = tree[v*2] + tree[v*2+1];
}
```

Реализация: range query

```
int query(int v, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return 0;
    if (ql <= l && r <= qr) return tree[v];
    push(v);                                  // проталкиваем перед запросом
    int mid = (l + r) / 2;
    return query(v*2, l, mid, ql, qr) +
           query(v*2+1, mid+1, r, ql, qr);
}
```

Задачи

- Дан массив из нулей и единиц. Подается m запросов двух видов:
  - инвертировать элемент с индексом i
  - узнать индекс k-ой единицы
- Дан массив чисел. Подается m запросов двух видов:
  - заменить элемент под индексом i на v
  - узнать минимальный индекс, с элементом, не меньше чем x
- Дана перестановка p. Найдите для каждого элемента количество инверсий.
- Дан массив длины $2n$ из чисел $1 \ldots n$, где каждое число встречается ровно два раза. Два вхождениия числа обозначают отрезок между двумя позициями. Задача: для каждого отрезка надо посчитать число пересекающихся с ним отрезков.